JARVIS

COMPLETE MASTER BLUEPRINT v3

End-to-end vision • Architecture • Flow diagrams • Security • Roadmap • Verification

Baseline: Core v1.1 commit 85abc21 | Prepared 11 October 2026

VISION → ARCHITECTURE → IMPLEMENTATION → VERIFICATION → CONTROLLED EVOLUTION

This is a living target specification. It distinguishes implemented/reported work from planned capabilities; it is not proof that future features exist.

How to Use This Document

This expanded edition includes detailed narrative, flow diagrams, current baseline, future capability catalog, dependencies, security controls, acceptance gates, and immediate actions. Update it after each milestone using verified code and audit reports.

Contents

• 1. Executive Summary

• 2. Vision and Product Goals

• 3. Baseline and Status Ledger

• 4. Complete System Map

• 5. Request-to-Response Flow

• 6. Mother Orchestrator, Planner, and Experts

• 7. Provider and Model Strategy

• 8. Memory, Conversation, and RAG

• 9. Core v1.2 Security and Ownership

• 10. API Contracts and Error Handling

• 11. Frontend and Interaction Design

• 12. Tool Runtime and Permission Gateway

• 13. Independent Verification and Software Engineering

• 14. Voice, Audio, and Multimodal Plan

• 15. Desktop, Browser, and Computer Control

• 16. Proactive Automation and Scheduling

• 17. Research, Finance, and Decision Support

• 18. Smart Home and Physical Devices

• 19. Defensive Cybersecurity and System Health

• 20. Tutor, Productivity, Travel, and Creative Tools

• 21. Deployment, Operations, and Recovery

• 22. Configuration Plan

• 23. Detailed Phase Roadmap and Exit Gates

• 24. Definition of Done and Governance

• 25. Immediate Next Actions

• Appendix A. Status Vocabulary

• Appendix B. Baseline References and Open Decisions

1. Executive Summary

JARVIS is a long-term personal AI operating system inspired by a capable conversational assistant. The goal is to unify conversation, planning, specialist reasoning, memory, research, software engineering, voice, controlled computer interaction, proactive monitoring, and selected integrations behind one coherent system.

The strategy is to build a secure and testable local-first foundation, then add capabilities in phases. Start as a modular monolith. Keep a mock provider for offline development. Add real models and integrations through explicit configuration. Avoid premature microservices, unrestricted autonomy, and features that cannot be verified.

JARVIS should be able to understand a request, produce a bounded plan, choose appropriate experts, use registered tools under policy, verify the results, and explain what succeeded, failed, or remains uncertain.

Baseline recorded for this document: Core v1.1, commit 85abc21. The v1.1 implementation and offline checks were reported complete. Core v1.2 security/ownership is the next planned milestone, not confirmed implemented. The recorded v1.1 live PostgreSQL integration suite was not run.

2. Vision and Product Goals

2.1 What JARVIS should eventually do

- Hold useful text and voice conversations and maintain explicit user-controlled context.
- Plan multi-step tasks and delegate them to specialist personas.
- Use registered tools and approved data sources, with clear permission boundaries.
- Remember selected information, retrieve it when relevant, and let the user inspect/edit/delete it.
- Help develop software: inspect repositories, propose changes, run tests, review diffs, and report evidence.
- Research public information and user-provided documents with citations and uncertainty statements.
- Offer desktop/mobile interfaces, voice, reminders, scheduled monitoring, and supported integrations.
- Assist with finance research, defensive cybersecurity, tutoring, travel, smart-home tasks, and creative workflows.
- Improve through evaluated, isolated, reviewed changes rather than silently rewriting itself.

2.2 Non-negotiable principles

- Truthfulness: never claim a tool or test ran unless it actually did.
- Privacy: minimize stored data and do not leak secrets or private content into logs.
- Human control: consequential actions require explicit approval.
- Least privilege: every tool receives only the access it needs.
- Modularity: providers, experts, tools, persistence, UI, and policy are replaceable behind contracts.
- Bounded execution: timeouts, retries, tool rounds, output sizes, concurrency, and budgets have limits.
- Recoverability: use backups, safe migrations, rollback or compensation where feasible.
- Evidence: important outputs need tests, citations, postconditions, or independent review.

2.3 What JARVIS is not

- Not an unrestricted autonomous agent or an unrestricted administrator.
- Not a system that silently sends messages, moves money, deletes important data, changes security settings, or controls dangerous devices.
- Not a multi-user service until a dedicated identity and tenancy model is designed and audited.
- Not public-network safe merely because it has a bearer token.
- Not permitted to fabricate citations, test results, provider status, or completed actions.

3. Baseline and Status Ledger

This is a documented project-history snapshot, not a fresh inspection of the local repository. Recheck Git status and rerun tests before release claims.

Milestone	Recorded scope	Status
v0.2 — 5fec820	Mother orchestrator foundation	Historically completed
v0.3 — fca5ba0	Optional OpenAI provider	Historically completed
v0.4 — 9fdb78c	Specialist intelligence and provider reliability	Historically completed
v0.5 — 86359d2	Structured specialist outputs and preflight validation	Historically completed
v0.6 — 77b27fd	Optional PostgreSQL persistence	Historically completed
v0.7 — 6bb4e7b	Conversation identity/history API	Historically completed

Milestone	Recorded scope	Status
v0.8 — ee25ca2	Opt-in PostgreSQL integration lifecycle	Historically completed
v0.9 — 24cb7eb	Explicit long-term memory, lexical search, bounded retrieval	Historically completed
v1.0 — 66e2e47	Memory retrieval quality, diagnostics privacy, lifecycle hardening	Historically completed
v1.1 — 85abc21	Memory workspace, category filtering, chat retrieval metadata	Committed; reported clean
v1.2	Single-owner auth, owner isolation, migration 0004	Planned; implementation not confirmed

3.1 Existing technical baseline

- Backend: Python, FastAPI, Pydantic, pytest.
- Frontend: React, TypeScript, Vite.
- Optional PostgreSQL persistence; lexical memory search, not semantic/vector search.
- Mock provider supports offline development; a real provider is optional.
- Memory creation is explicit, not automatically extracted from chat.
- Memory retrieval is disabled by default and capped at five records.
- Retrieved memories are untrusted reference context and cannot override system policy.
- Memory diagnostics are designed not to log memory text, query text, identifiers, or secrets.
- At v1.1, memory and conversation APIs lacked authentication and per-user ownership; v1.2 is intended to address this.
- Live PostgreSQL integration was not verified for the recorded v1.1 milestone.

4. Complete System Map

```
USER CHANNELS
[Web UI] [Desktop - future] [Mobile - future] [Voice - future]
|
v
API GATEWAY: request validation | authentication | rate/resource limits
|
v
MOTHER ORCHESTRATOR
Intent classification -> Planner -> Capability / Expert Registry
|
v
EXECUTION COORDINATOR (sequential by default; bounded parallel optional)
| | |
v v v
SPECIALIST EXPERTS TOOL RUNTIME MEMORY / KNOWLEDGE
Recommendation schemas/policies explicit memory CRUD
Analysis permission gateway lexical search -> future RAG
Validation timeouts/retries PostgreSQL optional
Risk / Compliance
Architecture / Knowledge
Coding / QA (future)
| | |
+------------------+-------------------+
v
INDEPENDENT VERIFIER -> RESULT SYNTHESIS -> SAFE AUDIT / RESPONSE
|
v
USER INTERFACE
```

The modular monolith keeps internal modules separated while avoiding the operational burden of distributed services. A service split is considered only when real scaling, reliability, or isolation requirements justify it.

5. Request-to-Response Flow

```
1. User submits request
|
2. Validate request schema and size
|
3. Authenticate/authorize if the selected operation requires it
|
4. Classify intent and determine whether planning is needed
|
5. Planner proposes bounded steps and required capabilities
|
6. Validate plan, schemas, limits, and permissions
|
7. Execute steps (sequential by default; safe parallel work optional)
|
8. Validate tool/expert outputs; handle timeout/failure/partial results
|
9. Independently verify important claims or action postconditions
|
10. Synthesize answer with evidence and uncertainty
|
11. Emit privacy-safe audit metadata
|
12. Return result and actual status to the UI
```

Request contract rules

- Use typed request and response schemas; confirm exact field names against the live code before editing.
- Clients never supply authoritative owner IDs, permissions, or claims that actions completed.
- Plan steps specify inputs, expected output, capability, permission level, timeout, and success criteria.
- Unknown tools, malformed plans, excessive rounds, and invalid parameters fail safely.
- Partial results must identify failed or skipped steps.

6. Mother Orchestrator, Planner, and Experts

6.1 Mother Orchestrator

- Accept the validated request and relevant conversation context.
- Classify intent and confidence; fall back to general inquiry with confidence 0.0 if classification fails.
- Choose direct response versus a multi-step plan.
- Select experts through a registry, not scattered hardcoded maps.
- Coordinate step ordering, bounded parallelism, tool rounds, retries, timeouts, and cancellation.
- Collect results and preserve source/provenance and failure details.
- Call a verifier when the task risk or acceptance criteria require it.
- Synthesize a clear, honest response and emit safe trace metadata.

6.2 Planner output

The Planner should return a structured plan: intent, steps, dependencies, selected expert/tool IDs, execution mode, expected outputs, constraints, and verification requirements. Runtime validation—not the language model—decides whether the plan is permitted.

6.3 Expert registry

Expert	Purpose	Typical output
RecommendationExpert	Compare options	Ranked options, trade-offs, recommendation
AnalysisExpert	Analyze data/evidence	Findings, reasoning summary, caveats
ValidationExpert	Check inputs/results	Pass/fail findings and corrections
RiskExpert	Identify failure modes	Risks, likelihood/impact, mitigations
ComplianceExpert	Check policy rules	Applicable requirements and gaps
ArchitectureExpert	Review technical designs	Components, dependencies, trade-offs
KnowledgeExpert	Retrieve/summarize knowledge	Grounded answer with provenance
Coding/QA experts (future)	Software changes and verification	Patch plan, tests, review findings

6.4 Configuration-driven registry

- experts.json concept: expert ID, role, prompt/persona, schemas, model preferences, and constraints.
- tools.json concept: tool ID, description, parameter schema, permission class, timeout, retry policy, and read/write classification.

- Model selection is configurable (for example LLM_MODEL).
- Sequential execution is default; parallel execution must respect dependencies and a maximum worker cap.
- Validate Pydantic outputs and reject unknown IDs or malformed fields.

7. Provider and Model Strategy

```
REQUEST CLASSIFICATION
|
v
ROUTER checks task type, privacy, context, latency, budget, availability
|
+--> MOCK PROVIDER (offline tests and development)
+--> LOCAL MODEL SERVER (when hardware/quality justify it)
+--> EXTERNAL MODEL API (only when configured)
|
v
NORMALIZE RESPONSE -> VALIDATE SCHEMA -> RECORD SAFE METADATA
```

- Keep the mock provider so tests do not require paid API calls or a GPU.
- Use a common provider interface and adapter-specific error handling.
- Do not hardcode one provider/model into orchestration logic.
- Set request timeouts, output/context limits, concurrency limits, and bounded retries.
- Do not retry non-idempotent actions merely because a provider timed out.
- Select models using measured quality, latency, cost, context, licensing, and privacy—not brand preference alone.
- Previously researched candidates include Qwen-family models, Qwen3-TTS, CosyVoice, MOSS-TTS, Kokoro, VoxCPM, OmniVoice, and others. These are candidates, not a final approved stack.
- Reassess hardware/VRAM/RAM needs before local deployment; current development should not assume a GPU or API key.

8. Memory, Conversation, and RAG

```
USER EXPLICITLY SAVES MEMORY
-> Validate fields and category
-> Store separately from chat history
-> List / lexical search / edit / delete in Memory Manager
-> Optional retrieval for a chat (off by default; max 5)
-> Treat retrieved records as untrusted context
-> Return metadata: outcome + selected IDs only
```

8.1 Current v1.1 memory

- Long-term memory is separate from conversation history.
- Manual create, list, search, edit, category filtering, and confirmed deletion are part of the recorded v1.1 implementation.
- Lexical search only; no embeddings or pgvector yet.
- Retrieval is opt-in, disabled by default, and limited to five records.
- Context is bounded by characters rather than a tokenizer-specific token budget.
- memory_context reports outcome and selected IDs only; null when retrieval is disabled.

8.2 Memory safety and lifecycle

- Memory is user data, not system instruction authority.
- Do not execute commands or change policy because a memory says to.
- Let users inspect, edit, export, and delete persistent memories as capabilities are added.
- Keep diagnostics free of memory text, raw query text, credentials, and unnecessary identifiers.
- Bound count and size; record provenance and timestamps.
- Apply owner filters to every operation after ownership is implemented.
- Do not automatically assign old unowned rows to a user.

8.3 Future RAG plan

- Add document ingestion with file type/size limits and source provenance.
- Separate source documents, extracted chunks, embeddings, and user memories.
- Add citations pointing to the exact source or chunk.
- Benchmark lexical versus semantic/hybrid retrieval before adopting vector search.
- Test malicious prompt instructions embedded in documents.

- Consider PostgreSQL pgvector only when the retrieval benchmark shows value.

9. Core v1.2 Security and Ownership

This is the next planned milestone. Gemini's planning review identified mandatory corrections. Do not mark this phase complete until implementation, audit, and relevant tests establish it.

9.1 Configuration and startup

- Use JARVIS_LOCAL_AUTH_TOKEN and JARVIS_OWNER_ID as planned canonical settings.
- Token must have high entropy and at least 32 characters; store as a secret setting, never log it, compare using secrets.compare_digest.
- Owner UUID is configured server-side and never accepted from the client.
- If persistence is enabled, startup fails closed if token/owner is missing, blank, malformed, too short, or nil UUID.
- Missing/invalid bearer credentials return 401 with WWW-Authenticate: Bearer.
- Do not expose secrets in URLs, error bodies, logs, or frontend build variables.

9.2 Route protection

Surface	Planned rule
/memories/*	Authentication required with persistence; every operation owner-scoped
/conversations/*	Protect persistent history reads, continuation, and writes
/chat	Protect whenever persistence is enabled, not only when retrieval is enabled
Stateless mock chat	May remain unauthenticated only if persistence is disabled and private stored data is not accessed
Health/readiness	Public only if sanitized and non-sensitive

9.3 Migration 0004

- Append-only migration adds nullable owner_id UUID without default.
- Composite index planned: (owner_id, created_at DESC, id ASC), partial for non-null owners if appropriate.
- Do not edit migrations 0001–0003.
- Legacy rows remain NULL/unowned and invisible through application APIs.
- No automatic ownership assignment and no HTTP claim endpoint.
- Any future claim must be an offline admin procedure with verified backup, dry-run count, owner check, and explicit confirmation.
- Application owner predicates are not equivalent to PostgreSQL row-level security.

9.4 Frontend credential lifecycle

- Keep bearer token in memory only; never localStorage, sessionStorage, cookies, URL, or Vite build variable.
- Provide password-style entry and explicit Lock Workspace.
- On 401 clear token and require re-entry.
- Send Authorization only to the validated configured API origin.
- Document same-origin XSS risk, plain HTTP risk, and local-only deployment assumptions.
- Do not claim single-owner authentication is a multi-user security boundary.

9.5 Required security tests

- Missing, malformed, wrong-scheme, empty, and incorrect credentials return 401 and challenge header.
- Invalid startup settings fail when persistence is enabled.
- Owner A cannot list/search/get/update/delete Owner B records.
- Unowned legacy rows are not visible or mutable through the API.
- Count queries and mutation SQL are owner-scoped and parameterized.
- Persistent chat/history are rejected without authentication.
- Token is never written to browser storage or sent to an external origin.
- Real PostgreSQL migration/isolation tests run on a disposable configured database before declaring integration verified.

10. API Contracts and Error Handling

Area	Contract
Chat	Preserve established fields; memory_context metadata only; null when retrieval disabled
Memory list	Exact category match, filtered count and rows, stable order and pagination
Memory CRUD	Owner-scope all operations; validate fields; sanitize errors
Auth	Bearer header; 401 plus WWW-Authenticate; no token in URL/body
Conversations	Persistent reads/writes authenticated; UUID is identifier, not authorization
Health	No secrets, DSNs, internal prompts, or private data
Errors	Stable error codes and safe messages; raw DB exceptions stay internal

- Confirm exact routes and schemas from source before implementation; this document defines behavior, not unverified endpoint inventory.
- Update backend and frontend contract tests together.
- Use additive API changes where safe; security may intentionally break old unauthenticated behavior.
- Do not return different messages that reveal whether a foreign-owned record exists.

11. Frontend and Interaction Design

11.1 Current interface direction

- React + TypeScript + Vite.
- Futuristic JARVIS aesthetic: dark charcoal/black, restrained cyan/blue accents, responsive layout and accessibility.
- Chat, sidebar, DEMO/MOCK status, loading/error/retry, suggested prompts, and Memory Manager.
- Unimplemented navigation remains visibly marked as coming soon.

11.2 UX standards

- Accessible labels, keyboard navigation, focus management, contrast, and reduced-motion consideration.
- Never mislabel mock responses as real model output.
- Show tool activity, permission prompts, and the final action result.
- Provide cancellation when safe and clear timeout/partial-failure states.
- Confirm permanent deletion and consequential changes.
- Never expose credentials in URLs, browser storage, analytics, or logs.

11.3 Future interface sequence

- Stabilize API/auth/policy before a desktop shell.
- Start voice with push-to-talk, visible microphone state, cancellation, and explicit audio consent.
- Prefer responsive mobile web first unless native device capabilities are required.
- Wake-word or always-listening modes require explicit opt-in and obvious controls.

12. Tool Runtime and Permission Gateway

```
MODEL PROPOSES TOOL CALL
-> Registry verifies tool exists
-> Validate schema, paths, domains, arguments
-> Authenticate caller and check policy
-> Classify impact / reversibility
-> Approval required? -- yes --> ask user; stop until approved
-> Execute with timeout, bounded retry, resource limits
-> Verify postcondition / state change
-> Record safe audit metadata
-> Report actual result
```

Tool class	Examples	Default
Read-only	Knowledge search, reports, system status	Allowed within scope
Draft-only	Draft email, patch, plan	No external send/write
Reversible write	Create file in approved workspace	Permission, audit, verify
High impact	Delete files, send message, account/money/device control	Explicit approval and stronger checks
Prohibited	Unauthorized access, credential theft, unbounded destructive command	Blocked

- Every tool declares schema, purpose, permission class, timeout, retry policy, idempotency behavior, and audit class.
- Model output never grants permission to itself.

- Use allowlists for paths, domains, commands, and integrations.
- Require explicit approval for irreversible or externally visible actions.
- Use rollback/compensation when possible; clearly state when an action cannot be undone.

13. Independent Verification and Software Engineering

```
USER TASK
-> Inspect repo and baseline
-> Write acceptance criteria and plan
-> Reproduce failure / baseline tests
-> Implement bounded change
-> Run targeted tests
-> Run regression suite + build + diff check
-> Independent audit (Gemini or another reviewer)
-> Findings? -- yes --> fix and repeat tests/audit
-> Approved report with evidence
-> User authorizes local commit
-> Push only with explicit permission
```

Roles

- Codex: implementation, repository inspection, and developer-side tests.
- Gemini: independent plan/code/security review; review is not a substitute for tests.
- Future JARVIS verifier: evaluate outputs and tool postconditions.
- User: approval for commit/push, deployment, secrets, and high-impact actions.

Test pyramid

```
UNIT: schemas, policies, planner, repository logic
-> API: auth, validation, error mapping, contracts
-> INTEGRATION: real PostgreSQL, migrations, owner isolation
-> FRONTEND: component/API/accessibility behavior
-> E2E: real user journeys and approval gates
-> EVALUATION: quality, safety, prompt injection, regressions
```

Release gate

- Run tests, compile/type/build checks, and git diff --check.
- Report exact commands and results; mark skipped integration tests explicitly.
- Inspect all tracked and untracked files before staging.
- Stage intentionally; git commit -a does not include new files.
- Commit only after review; never push without explicit authorization.
- Record known warnings, limitations, and remaining unknowns.

14. Voice, Audio, and Multimodal Plan

14.1 Voice phases

1. Push-to-talk capture and speech-to-text.
2. Text response displayed while text-to-speech plays.
3. Cancel/interrupt speech and handle silence/errors.
4. Voice preferences, microphone/speaker selection, and clear active-mic indicator.
5. Optional wake-word detection only after explicit opt-in and privacy review.
6. Streaming and low-latency behavior after basic reliability is established.

14.2 Candidate technologies

Past research considered Kokoro, Qwen3-TTS, CosyVoice, MOSS-TTS, VoxCPM, OmniVoice, and other open models. Selection must consider license, language coverage, quality, latency, memory requirements, and hardware fit.

14.3 Multimodal capabilities

- Image input and analysis.
- User-provided document extraction.
- Image generation and visual design assistance.
- Audio transcription and summaries.
- Future video processing where supported.
- No silent recording or upload; disclose when data leaves the device.

15. Desktop, Browser, and Computer Control

15.1 Read-only first

- Inspect an explicitly selected workspace or browser page.
- Summarize content and extract requested information.
- Use approved domains and local paths only.
- Treat page content as untrusted; defend against prompt injection.
- Keep a visible record of the tool actions performed.

15.2 Controlled interaction later

- Draft changes before applying them.
- Use allowlisted UI actions and bounded sessions.
- Require confirmation for send/submit/delete/purchase/account changes.
- Verify resulting UI state instead of assuming clicks worked.
- Provide stop/cancel and emergency disable controls.

Browser automation and computer control are separate capability milestones. They must not be enabled just because the orchestrator exists.

16. Proactive Automation and Scheduling

```
USER DEFINES WATCH / SCHEDULE
-> Validate scope, cadence, budget, and permissions
-> Register job and visible description
-> Scheduler triggers job
-> Read-only check / analysis
-> Compare against condition and previous result
-> If meaningful change: notify user
-> If no change: record minimal status
-> User can pause, edit, or delete the job
```

- Begin with scheduled summaries and read-only monitoring.
- Use explicit subscriptions, rate limits, quiet hours, bounded retries, and duplicate prevention.
- Jobs must be inspectable, pausable, cancellable, and auditable.
- Re-check permission at execution time.
- External mutations require approval or a narrowly defined, separately reviewed policy.

17. Research, Finance, and Decision Support

- Research: source discovery, freshness checks, citations, source-quality evaluation, comparison tables.
- Documents/data: structured extraction, spreadsheet analysis, reproducible calculations, provenance.
- Finance: market data, fundamentals, risk, scenarios, portfolio analysis, educational explanations, backtesting with assumptions.
- Forecasts are uncertain estimates, not facts. Address data leakage, survivorship bias, and transaction costs in backtests.
- Actual trades, payments, or transfers are separate high-impact operations and require explicit authorization and provider-specific controls.
- Keep API credentials isolated and avoid storing financial secrets in conversational memory.

18. Smart Home and Physical Devices

- Start by reading device/sensor state through an approved integration.
- Add low-risk reversible controls only after opt-in and device-level policy.
- Locks, alarms, cameras, garage doors, heating, and safety-critical devices need stronger controls and confirmation.
- Do not infer permission for a high-impact action from conversational context alone.
- Verify the resulting device state and report failures or unavailable devices.
- Provide a global pause/disable mechanism for device actions.

19. Defensive Cybersecurity and System Health

- Monitor authorized CPU, memory, disk, services, and logs.
- Review configurations and dependency vulnerabilities.
- Inspect authorized assets and local network configuration.
- Detect anomalies and recommend evidence-based remediation.

- Keep scope explicit; no unauthorized intrusion, credential harvesting, or destructive remediation.
- Require approval before deleting, quarantining, disabling security controls, or changing production configurations.

20. Tutor, Productivity, Travel, and Creative Tools

Tutor

- Explain concepts at the user's current level.
- Create exercises, quizzes, and progress summaries.
- Distinguish demonstrated mastery from assumed mastery.

Productivity

- Tasks, reminders, calendar summaries, and approved scheduling.
- Draft emails/documents; sending requires confirmation.
- Recurring reports with pause/stop controls.

Travel

- Research destinations, routes, and options using current sources.
- Build itineraries and compare trade-offs.
- Bookings and purchases require explicit confirmation and rechecking details.

Creative

- Writing, brainstorming, image/audio generation, and design assistance.
- Use explicit user requests and respect rights, licensing, and safety limits.

21. Deployment, Operations, and Recovery

21.1 Deployment progression

1. Local development with mock provider.
2. Local persistence on a configured PostgreSQL instance.
3. Local packaged application with health checks and documented configuration.
4. Optional secure remote access only after a separate threat model, transport security, access controls, and operational review.

21.2 Observability

- Use execution IDs, durations, outcome codes, counts, and sanitized error categories.
- Never log bearer tokens, provider keys, credential-bearing DSNs, raw memory content, or private prompts by default.
- Sanitize exceptions returned to users.
- Define retention and restrict diagnostics.

21.3 Backup and recovery

- Document what data is stored and how to export/delete it.
- Back up before migrations or ownership changes; verify restore in a disposable environment.
- Use append-only migrations; do not rewrite old migration history.
- Integration tests may only drop databases they created and identified as disposable.
- Maintain runbooks for provider outage, DB outage, token rotation, migration failure, restore, and suspected credential compromise.

22. Configuration Plan

Setting/concept	Purpose	Secret?
LLM_MODEL	Select model	No
Provider credential	External provider authentication	Yes
Persistence flag	Enable PostgreSQL storage	No
Database URL	Connect to PostgreSQL	Usually sensitive
JARVIS_LOCAL_AUTH_TOKEN	Planned bearer credential	Yes
JARVIS_OWNER_ID	Planned stable owner UUID	No
Memory retrieval flag	Opt in to retrieval	No
Integration-test opt-in	Enable live integration suite	No
Test admin URL	Provision disposable test DB	Treat as sensitive
PGPASSFILE	Password file	Yes; protect ACLs

Confirm exact setting names in code before use. Never commit real secrets. Keep .env ignored, use placeholders in .env.example, and never put backend credentials in Vite build variables.

23. Detailed Phase Roadmap and Exit Gates

Phase	Scope	Exit gate
A. Core runtime	API, schemas, mother orchestrator, provider abstraction	Mock works; contracts tested
B. Specialist execution	Planner, registry, structured outputs	Invalid plans rejected; partial failures tested
C. Persistence	Optional PostgreSQL and conversations	Migration lifecycle tested
D. Memory	Explicit CRUD, lexical search, bounded retrieval	Privacy-safe diagnostics and bounds
E. Memory UI	Search/edit/delete/context indicator	Accessible flows tested
F. Security v1.2	Single-owner auth, owner isolation, migration 0004	Auth tests plus real DB migration/isolation tests
G. Evaluation harness	Golden tasks, quality/safety/adversarial tests	Baseline and regression gates
H. RAG	Document ingestion, provenance, retrieval evaluation	Citation and injection tests
I. Coding agent	Repo exploration, patch, tests, review	No automatic commit/push
J. Browser/desktop	Read-only first; controlled interaction later	Allowlist, approval, postcondition checks
K. Voice	Push-to-talk, STT, TTS	Consent and latency/error tests
L. Automation	Scheduled summaries and condition monitoring	Pause/stop, limits, audit
M. Integrations	Calendar, email, files, GitHub, smart home	Least privilege, scoped credentials
N. Deployment	Packaging, monitoring, backup/restore	Reproducible install and recovery drill
O. Controlled improvement	Suggest, sandbox, test, audit	Human approval before release

The order can evolve based on evidence, but dependencies must be respected: secure identity before private integrations; evaluation before broad autonomy; recovery before high-value data; permission gateway before mutating tools.

24. Definition of Done and Governance

- Every milestone has explicit scope, dependencies, acceptance criteria, and test plan.
- Implementation report lists files changed and actual commands/results.
- Documentation is updated: README, ARCHITECTURE, HANDOFF, ROADMAP, and this blueprint where appropriate.
- Skipped tests and known limitations are written down.
- Security-sensitive work receives independent review.
- Complete diff and untracked files are inspected before staging.
- No commit or push is implied by a passing test suite.
- No push without explicit user authorization.
- Self-improvement follows: propose -> isolated branch/sandbox -> test -> independent audit -> human approval -> commit/release.

25. Immediate Next Actions

1. Confirm repository is at commit 85abc21 with a clean working tree before starting v1.2.
2. Implement fail-closed settings and bearer authentication.
3. Protect every persistence-backed /memories/*, /conversations/*, and /chat operation.

4. Add migration 0004 with nullable owner_id; leave legacy rows unowned and inaccessible.
5. Scope every memory operation and retrieval to the configured owner.
6. Implement frontend in-memory token entry, lock, 401 recovery, and API-origin containment.
7. Run offline tests/build; run opt-in PostgreSQL migration and owner-isolation tests against a safe disposable database.
8. Obtain independent Gemini audit and resolve blocking findings.
9. Update documentation with actual results and commit hash.
10. Commit locally only after review; push only after explicit authorization.

Do not start vector search, broad computer control, proactive autonomy, or public deployment before security and verification gates are satisfied.

Appendix A. Status Vocabulary

Status	Meaning
Implemented (reported)	A report says it exists; verify before stronger claims
Committed	A Git commit exists; not necessarily pushed/deployed
Offline tested	Unit/frontend checks passed; does not prove real PostgreSQL behavior
Integration verified	Test ran against the real configured dependency
Planned	Design only; not implemented
Blocked	A prerequisite is unmet
Not verified	Evidence is insufficient; do not claim it works

A diagram is a target design, not proof of implementation. Update this status ledger after every milestone.

Appendix B. Baseline References and Open Decisions

- Latest recorded baseline: Core v1.1 commit 85abc21.
- Earlier commits: v0.2 5fec820; v0.3 fca5ba0; v0.4 9fdb78c; v0.5 86359d2; v0.6 77b27fd; v0.7 6bb4e7b; v0.8 ee25ca2; v0.9 24cb7eb; v1.0 66e2e47.
- v1.1 checks were reported: 147 Python tests, 25 frontend tests, build, compileall, and diff check passed.
- Live PostgreSQL integration was not run for the recorded v1.1 milestone.
- Open decisions: final local model, vector store (only if justified), voice stack, desktop/mobile framework, approved integration providers, remote-access design, retention defaults, and production deployment model.